AI Legal Case Operations Platform

Software Requirements Specification & Technical Architecture

Version 4.0 — Multi-Agent / Agentic RAG Edition

A single-lawyer private workspace for evidence-grounded legal document research, with resilient ingestion for messy real-world files, hybrid retrieval, coordinated specialist agents, source citations, and explicit uncertainty handling.

Architecture baseline

Next.js + React + TypeScript frontend, separately deployed; Python FastAPI backend under a standalone backend/ directory; Supabase Auth, Postgres and private Storage; Pinecone vector search; CPU-based sentence-transformer embeddings; OpenAI for language-model reasoning; LangChain and LangGraph for orchestration.

Important engineering promise: the system is designed to detect, isolate, repair where safe, and report data-quality problems. No architecture can guarantee that every damaged, ambiguous, handwritten, or low-quality source will be interpreted perfectly. The system must abstain, request review, or disclose uncertainty when confidence is inadequate.

Prepared as an implementation-oriented specification. The agent roles below are explicit design requirements for v4; they were not all present as implemented agents in v3.

Document control and how to use this specification

This v4 document extends the previous SRS and architecture with an explicit agentic RAG design. It retains the independent frontend/backend deployment model, single-user scope, Supabase data/auth/storage, Pinecone vector index, local CPU embeddings, OpenAI LLM, and a Python backend that can be deployed independently.

- **Normative requirements:** statements using MUST or MUST NOT are required for a conforming implementation.
- **Recommended controls:** SHOULD means the control is strongly recommended unless a documented reason justifies a different choice.
- **Implementation sequence:** build a reliable non-agent document pipeline and retrieval first; then add LangGraph orchestration and specialist agents behind measurable evaluation gates.
- **Not legal advice:** the product assists a lawyer with evidence organization and research. It does not autonomously make legal decisions, file documents, contact opposing parties, or replace professional judgment.

1. Executive summary

The platform is a private legal-case workspace for one lawyer. The lawyer uploads contracts, pleadings, correspondence, PDFs, scans, images, and other case materials, then asks questions across those materials. The platform processes documents, retains source/page provenance, retrieves evidence through hybrid search, and uses a coordinated graph of specialist agents to produce grounded answers and reports.

The core pattern is **Agentic RAG**: retrieval-augmented generation embedded inside a controlled workflow. Agents can plan searches, retrieve evidence, extract facts, compare documents, identify gaps or contradictions, and validate citations. LangGraph owns the workflow state, conditional routing, retries, budgets, and stop conditions. The design is not an unconstrained group chat between LLMs.

1.1 Product goals

- Provide answers grounded in the user's private case documents, with source title and page/section citations wherever available.
- Handle dirty data: scanned PDFs, mixed-quality OCR, rotated pages, skew, tables, duplicated files, inconsistent filenames, missing metadata, encoding errors, fragmented text, and conflicting versions.
- Separate document facts, inferred interpretations, and unsupported assumptions.
- Detect insufficient evidence and contradictions; retry retrieval when useful and otherwise explain the gap.
- Keep frontend and Python AI backend independently deployable and configurable.
- Make every stage testable, observable, repeatable, and auditable without exposing private document content in ordinary logs.

1.2 Non-goals

- No multi-tenant SaaS, organizations, teams, memberships, or role-management product in this version.
- No autonomous legal advice, legal representation, court filing, e-signature, external email sending, or actions with legal consequences.
- No guarantee that OCR, extraction, legal interpretation, or citations are always correct.
- No premature microservices: each deployable unit is a modular application. The frontend and backend are separate deployments, but backend agents are modules in one Python service.
- No GPU dependency for the embedding model. Local CPU embeddings are the default; throughput must be measured on the target CPU.

2. Users, assumptions, and system boundaries

2.1 User model

One authenticated lawyer uses a private workspace. Supabase Auth provides identity; authorization checks ensure the authenticated user can access a case and every related document. The schema may include user_id ownership fields for defense in depth, but it does not require organization/team tables.

2.2 System boundary

Component	Responsibility	Deployment
Next.js frontend	User interface, auth-aware screens, upload initiation, server routes/actions, request proxy	Vercel or equivalent
Python FastAPI backend	Ingestion, parsing/OCR, embeddings, indexing, retrieval, agentic graph, answer generation, evaluation hooks	Independent Python deployment
Supabase Auth + Postgres	Identity, case/document metadata, audit records, job state, permissions	Managed Supabase
Supabase private Storage	Original uploads and derived artifacts where appropriate	Private bucket; signed URLs or backend-mediated access
Pinecone	Vector index for searchable chunks	Managed vector service
OpenAI API	LLM calls for extraction, planning, analysis, synthesis, validation where configured	External API; minimize transmitted data
Local embedding model	Dense vector generation using sentence-transformers/all-MiniLM-L6-v2	Backend CPU; model cache local to backend

3. Architecture overview

```
Browser
  |
  | HTTPS, Supabase Auth session
  v
Next.js frontend / server routes (independent deployment)
  |  - checks session and case access
  |  - forwards access token to backend
  |  - uses server-only AI_BACKEND_BASE_URL
  v
Python FastAPI backend (independent deployment)
  |-- Auth token verification + case/document authorization
  |-- Ingestion pipeline and durable job state
  |-- LangGraph Agentic RAG orchestrator
  |     |-- Query/Task Router
  |     |-- Retrieval & Research Agent
  |     |-- Document/Facts Analyst
  |     |-- Contradiction & Gap Analyst
  |     |-- Evidence/Citation Verifier
  |     |-- Answer/Report Synthesizer
  |     `-- Quality Gate / retry or abstain
  |-- CPU embeddings -> Pinecone
  |-- Supabase Postgres + private Storage
  `-- OpenAI API (LLM calls)
```

Default request path: browser calls the Next.js app using the public app URL; the Next.js server calls Python using AI_BACKEND_BASE_URL. The backend independently validates the bearer token and checks case access. This avoids browser-to-Python CORS in the default architecture.

3.1 Cross-service URLs

- Use absolute base URLs from environment variables for cross-service calls; do not hard-code production hostnames.
- Frontend public variable: NEXT_PUBLIC_APP_URL.
- Next.js server-only variable: AI_BACKEND_BASE_URL. Never prefix server secrets or private backend URLs with NEXT_PUBLIC_.

- If direct browser-to-Python calls are added later, configure NEXT_PUBLIC_AI_API_BASE_URL and a strict allowlist CORS policy; do not use wildcard origins with credentials.
- Validate URL schemes and hosts at startup. Production should use HTTPS.

4. Repository layout and independent deployment

```
project-root/
  frontend/                       # Next.js + React + TypeScript
    src/
    public/
    .env.example
    package.json
  backend/                        # standalone Python application
    app/
      api/
      core/
      auth/
      db/
      storage/
      ingestion/
        detectors/
        pdf/
        ocr/
        parsing/
        normalization/
        deduplication/
        quality/
      embeddings/
      indexing/
      retrieval/
      agents/
        state.py
        graph.py
        router.py
        research_agent.py
        document_analyst.py
        contradiction_agent.py
        evidence_verifier.py
        answer_synthesizer.py
        prompts/
      jobs/
      evaluation/
      observability/
    tests/
    scripts/
    requirements.txt or pyproject.toml
    Dockerfile
    .env.example
    .cache/                       # local runtime only; gitignored
    .tmp/                         # local runtime only; gitignored
    .venv/                        # local virtual environment; gitignored
  supabase/
    migrations/
  docs/
  .gitignore
  README.md
  AGENTS.md
```

All Python code, Python dependencies, backend container configuration, model-cache settings, temporary-file settings, backend tests, and backend deployment instructions belong under backend/. Supabase migrations can remain at the repository root because they define shared database infrastructure. Runtime cache, uploads, and temporary files must not be committed.

5. Agentic RAG: design principles

5.1 What makes it agentic?

A normal RAG chain usually performs a mostly fixed sequence: retrieve chunks, place them in a prompt, and generate an answer. This system uses a graph that can choose among actions based on the task and evidence state: search again with a rewritten query, inspect a specific document, compare versions, request structured fact extraction, validate citations, or stop with an insufficient-evidence response.

- **Bounded autonomy:** agents may choose among approved tools and routes only. They cannot execute arbitrary code, access unrelated cases, or perform external side effects.
- **Shared typed state:** agents read and update a defined graph state rather than chatting freely.
- **Evidence-first:** conclusions must be tied to source spans, not merely to another agent's unsupported claim.
- **Explicit budgets:** maximum graph steps, retries, LLM tokens, tool calls, latency, and spend per request.
- **Deterministic controls:** authorization, schema validation, citation existence, deduplication, and permission checks use code wherever possible.
- **Human control:** the lawyer reviews high-impact interpretations; the system never files or sends anything on its own.

5.2 Proposed specialist roles

Role	Primary task	Allowed inputs/tools	Output contract
Coordinator / Task Router	Classify request, define subquestions, select graph path	User question, authorized case metadata, available tool registry	Task type, bounded plan, subquestions, budget
Research & Retrieval Agent	Gather relevant evidence and improve searches when recall is weak	Hybrid retrieval, metadata filters, query rewrite, document list	Ranked source spans, search traces, coverage notes
Document/Facts Analyst	Extract structured facts from selected passages or documents	Authorized page text, layout/table data, extraction schema	Facts with exact source spans, dates/entities, confidence/quality flags
Contradiction & Gap Analyst	Find inconsistent dates, conflicting statements, missing evidence	Facts, document versions, source spans	Contradiction records, unresolved questions, evidence gaps
Evidence & Citation Verifier	Check that each factual claim is supported by source spans	Draft claims, source IDs/page anchors, deterministic validators	Supported/unsupported/partial claim labels, citation corrections
Answer / Report Synthesizer	Compose readable answer from verified evidence	Verified claims, caveats, user question	Structured answer, citations, limitations, next questions
Quality Gate (mostly deterministic)	Decide accept, retry, request clarification, or abstain	Coverage metrics, verifier results, retry budget	Final route and reason codes

Not every role must be an independent LLM call. The quality gate, permission checks, citation existence checks, deduplication, and basic date/metadata validation should be deterministic code. Start with fewer calls and add an LLM agent only when evaluation shows a measurable benefit.

5.3 Agent responsibilities and boundaries

- The Router plans but does not make unsupported factual claims.
- The Research Agent retrieves evidence; it does not decide that a legal conclusion is true merely because a passage ranks highly.
- The Facts Analyst extracts what the source says and preserves uncertainty; it must not silently normalize a disputed date or name.
- The Contradiction Analyst flags conflicts rather than selecting a winner without a documented rule.
- The Verifier checks source support; it cannot create new evidence.

- The Synthesizer may only use verified claims and must retain caveats and source links.
- No agent may change access controls, modify originals, or trigger external actions.

6. LangGraph workflow and state

6.1 Query-time graph

```
START
  -> authenticate + authorize case
  -> classify request / route task
  -> plan subquestions (bounded)
  -> hybrid retrieve per subquestion
  -> assess retrieval coverage
       |-- weak coverage and retries remain -> rewrite query -> retrieve
       |-- enough evidence -----------------> extract/organize facts
       `-- still weak ----------------------> mark evidence gap
  -> detect conflicts / missing facts
  -> draft claims from evidence
  -> verify claim-to-source support + citation anchors
       |-- material failures and budget remains -> targeted retrieval/revise
       |-- unresolved material failures --------> abstain/qualify
       `-- acceptable --------------------------> synthesize answer
  -> final schema/safety/citation gate
  -> audit metadata + response
END
```

6.2 Typed state (conceptual)

```
AgenticRAGState = {
  request_id, user_id, case_id, question, task_type,
  authorized_document_ids,
  plan: [subquestion, purpose, status],
  retrieval_results: [source_span],
  extracted_facts: [fact_with_provenance],
  contradictions: [conflict_record],
  evidence_gaps: [gap_record],
  draft_claims: [claim_with_citations],
  verification_results: [claim_status],
  retries_used, llm_calls_used, token_budget, deadline,
  final_answer, final_status, reason_codes
}
```

Use Pydantic models or equivalent typed schemas for state payloads and tool outputs. Store only what is needed. Do not put full document text or secrets into routine logs. Large payloads should be referenced by IDs or persisted artifacts rather than passed repeatedly between nodes.

6.3 Stop conditions

- The request is answered to the configured evidence threshold.
- The system has completed allowed targeted retries and still lacks evidence.
- A required document is unavailable, corrupt, password-protected, or not parseable.
- The request is outside scope or lacks a necessary case/document reference.
- The graph reaches a step, latency, token, tool-call, or cost budget.
- An authorization or safety control fails; fail closed and do not continue retrieval.

7. Dirty-data strategy: ingestion quality and recovery

Dirty data is a first-class system requirement, not an afterthought. The pipeline must preserve the original file, detect quality problems, choose a suitable processing path, and record what was and was not recovered. Never overwrite the original or pretend a lossy extraction is complete.

7.1 Expected dirty-data classes

Problem	Detection / signal	Safe response
Scanned or image-only PDF	Very low text density; page images contain text	Render pages, OCR, retain page-level OCR confidence and coordinates
Rotated/skewed pages	Orientation detector, skew estimate	Deskew/rotate derived image; retain original and transformation metadata
Blur, low contrast, noise	Image-quality metrics, OCR confidence	Apply bounded preprocessing variants; compare OCR outputs; flag low confidence
Mixed digital and scanned pages	Per-page text density varies	Choose native text extraction or OCR page by page
Broken PDF / unreadable file	Parser exceptions, invalid structure	Quarantine job, record error, allow re-upload or manual review
Password-protected document	Encrypted-file detection	Request authorized password or unlocked copy; do not bypass controls
Tables, columns, headers/footers	Layout segmentation and reading-order checks	Use layout-aware extraction; preserve table structure and cell provenance
Bad encoding / weird Unicode	Replacement characters, control chars, normalization checks	Normalize Unicode conservatively; preserve raw extraction for audit
Duplicate uploads / renamed copies	Cryptographic hash and near-duplicate similarity	Deduplicate processing where safe; retain each upload record and lineage
Conflicting versions	Metadata, hashes, dates, text similarity, version labels	Keep separate versions; show conflict and provenance, never merge silently
OCR errors in names, numbers, dates	Low confidence, inconsistent entities, checksum/format rules	Flag and compare against image/source; do not silently invent correction
Missing metadata / poor filenames	Blank or unreliable title/date/type	Assign stable ID; infer suggestions with provenance; ask user to confirm material metadata
Truncated or repeated text	Page boundary checks, repeated n-grams, length anomalies	Flag extraction; retry with alternative parser or split strategy
Handwriting or stamps/signatures	Low OCR confidence or handwriting detector	Use specialized OCR only if enabled; otherwise flag for manual review
Very large or malicious file	Size/page limits, parser timeout, malware scanning where available	Reject/quarantine safely; sandbox parsers; enforce resource limits

7.2 Ingestion quality pipeline

1. **Receive safely:** validate authenticated user, case access, file size, extension, MIME type, magic bytes, and allowed formats. Generate a server-side random object key; never trust user-supplied path names.
2. **Quarantine and scan:** store original in private storage; apply malware scanning if available; do not execute embedded content. Use parser sandboxing and CPU/memory/time limits.
3. **Fingerprint:** calculate cryptographic hash, file size, page count, MIME detection, and upload timestamp. Hash is for deduplication/integrity, not access authorization.
4. **Classify pages:** detect native text versus image-only pages, orientation, image quality, likely tables/columns, and encryption/corruption.
5. **Extract:** use native PDF/text parsers first when reliable; OCR only where needed. For mixed PDFs, route per page rather than assuming one method fits the whole file.
6. **Normalize conservatively:** normalize Unicode, whitespace, hyphenation and repeated headers/footers only when rules are safe. Preserve raw extraction and transformation logs.
7. **Validate output:** check page coverage, text density, extraction errors, reading order, OCR confidence, table presence, and suspicious truncation.
8. **Repair or retry:** use bounded alternative extraction/OCR settings. Compare output quality, do not endlessly retry.

9 **Mark quality status:** ready, ready_with_warnings, needs_review, failed, or password_required. Make status visible to the lawyer.

10 **Chunk and index:** only index usable text; attach quality flags and source anchors to each chunk. Do not present low-confidence OCR as certain evidence.

11 **Audit:** record pipeline version, parser/OCR versions, transformations, warnings, and timestamps without logging sensitive text unnecessarily.

7.3 Quality score and routing

Use a transparent, explainable quality profile rather than a mysterious single score. Signals may include text coverage by page, OCR confidence, reading-order checks, duplicate/repeated text ratio, parser errors, table detection, and citation-anchor availability. A weighted score can prioritize review, but it must not be treated as a calibrated probability of correctness.

Status	Example rule	Search / answer behavior
ready	Text coverage and anchors acceptable; no material extraction warnings	Normal retrieval
ready_with_warnings	Some pages low-confidence or tables imperfect	Search allowed; surface warning and page-level confidence
needs_review	Important pages missing/garbled or extraction ambiguous	Allow limited use with prominent caveat; request cleaner upload for high-impact questions
failed	Corrupt, unsupported, empty, or repeatedly failed extraction	Do not index as usable evidence; show actionable error
password_required	Encrypted file cannot be opened	Request unlocked file or user-provided password through a secure flow

7.4 OCR and parser design

- Prefer native PDF text extraction when the PDF contains reliable text; use OCR for image-only or poor native-text pages.
- Retain page number, bounding boxes where available, extraction method, OCR engine/version, and confidence values.
- Use OpenCV for image preprocessing such as orientation correction, deskew, denoising, contrast enhancement, and thresholding only when evaluation shows improvement.
- Preserve layout for multi-column pages, tables, footnotes, stamps, and headers. Flattening everything into a single string can destroy meaning.
- For tables, retain rows/cells or a structured representation plus a text representation. Cite the page and, where possible, table/row context.
- Never silently repair a legal party name, amount, date, section number, or negation. Store a suggested correction separately and mark it as unverified.
- Treat OCR confidence as a signal, not proof: a high confidence score can still be wrong, especially for legal names and numbers.

8. Chunking, embeddings, and indexing

8.1 Chunking policy

Use structure-aware chunking, not blind fixed-size splitting alone. Keep headings, clauses, numbered paragraphs, tables, and page boundaries when possible. An initial experiment can begin around 150–220 tokens per chunk with 20–40 tokens of overlap, then be tuned using legal-question evaluation rather than treated as a universal optimum.

- Attach case_id, document_id, upload/version ID, page start/end, section heading, chunk index, extraction quality, source span/coordinates, and content hash.
- Avoid chunks that combine unrelated clauses or versions. Keep table rows together where practical.
- Use parent-child retrieval or neighboring-chunk expansion for clauses whose meaning depends on surrounding text.
- Do not embed metadata or boilerplate in a way that overwhelms meaningful legal text.
- Keep an index schema version and embedding model/version to support re-indexing.

8.2 Embeddings

Default model: sentence-transformers/all-MiniLM-L6-v2 on CPU, typically producing 384-dimensional embeddings. Confirm the actual model output dimension at startup and ensure it matches the Pinecone index. Cache model files locally under backend/.cache/huggingface; cache is not committed to Git.

- Run embeddings in bounded batches and measure CPU/RAM usage.
- Keep model loading out of per-request hot paths; initialize once per worker where feasible.
- Record embedding model and normalization settings for every index version.
- If changing the embedding model, use a new index/version or a controlled re-index plan; never mix incompatible vectors.

8.3 Pinecone and cross-store consistency

Postgres is the source of truth for case/document/job state. Pinecone is a derived search index. Each vector ID must map to a durable chunk record or reconstructable chunk metadata. Use idempotent upserts and deletion workflows.

- Write extraction/chunk records and an indexing job state to Postgres before/around vector upserts using a recoverable state machine.
- Use idempotency keys for upload and indexing jobs; retry transient failures with exponential backoff and jitter.
- Mark indexing as complete only after expected vectors are confirmed or a reconciliation policy accepts partial indexing.
- If the upload is replaced or deleted, remove corresponding vectors and mark the document state consistently.
- Run periodic reconciliation for orphaned vectors, missing vectors, and stale index versions.
- Do not assume a database transaction can atomically commit both Postgres and Pinecone; design for retries and repair.

9. Retrieval: hybrid search and evidence assembly

9.1 Retrieval flow

1. Classify the question (fact lookup, chronology, compare versions, summarize document, issue analysis, or broad case question).
2. Break complex questions into a small, explicit set of subquestions.
3. Run dense semantic retrieval and lexical/keyword retrieval with case/document filters.
4. Fuse rankings using a tested method such as Reciprocal Rank Fusion; avoid assuming raw scores from different retrievers are directly comparable.
5. Apply metadata filters, duplicate suppression, and source diversity rules while preserving highly relevant passages.
6. Optionally rerank a bounded candidate set if evaluation shows measurable improvement and latency/cost are acceptable.
7. Expand selected chunks with adjacent context when clause boundaries or cross-references require it.

8 Return evidence packets with exact source IDs, page anchors, extraction quality, retrieval score/rank, and text spans.

9 Measure coverage and decide whether to answer, retry targeted retrieval, ask a clarification question, or abstain.

9.2 Query rewriting and multi-hop research

The Research Agent may rewrite a query to include synonyms, section names, party aliases, dates, or alternate terms found in the case. It must preserve the original question and log the rewrite strategy. Multi-hop search can follow references such as “subject to clause 12” or a date found in another document, but every hop must remain within authorized case scope and a strict step budget.

9.3 Retrieval quality controls

- Use case_id and authorized document IDs as server-enforced filters; never rely on prompt instructions to enforce access.
- Treat document text as untrusted data. Ignore instructions embedded in uploaded files that attempt to change system behavior.
- Deduplicate near-identical chunks and repeated headers, but retain source links to every relevant document/version.
- Keep conflicting versions visible; do not collapse them into one synthetic “truth.”
- If retrieval is weak, the system should say which evidence is missing rather than fabricate an answer.

10. Agent contracts and prompts

10.1 Shared evidence object

```
EvidenceSpan = {
  evidence_id: str,
  case_id: str,
  document_id: str,
  document_version_id: str,
  title: str,
  page_start: int | None,
  page_end: int | None,
  section_path: list[str],
  quote_or_span: str,
  extraction_method: str,
  quality_status: str,
  ocr_confidence: float | None,
  source_anchor: str,
  retrieval_method: str,
  retrieval_rank: int
}
```

10.2 Research Agent contract

Input: original question, subquestions, authorized case/document scope, prior retrieval results, remaining budget. **Output:** evidence spans, queries attempted, coverage assessment, gaps, and recommended next search. The agent cannot broaden access scope or claim that a retrieved passage proves a conclusion.

10.3 Document/Facts Analyst contract

Input: bounded evidence packet or selected document pages, schema for requested facts. **Output:** typed facts with exact source span, page, entity/date/amount, extraction quality, and uncertainty. Every fact must reference evidence IDs. If text is ambiguous, return multiple candidates or unknown rather than guessing.

10.4 Contradiction & Gap Analyst contract

Input: facts and evidence from relevant documents/versions. **Output:** conflict records containing the conflicting propositions, source spans, possible reasons (different dates, versions, perspectives, OCR ambiguity), and a resolution status. It must not choose a preferred fact unless a documented, explainable rule supports that choice.

10.5 Evidence Verifier contract

Input: draft factual claims and their citations. **Output:** supported, partially supported, unsupported, contradicted, or citation-invalid. Use deterministic checks to ensure cited IDs exist, belong to the authorized case, and have valid page anchors. LLM-based entailment is a second opinion, not a replacement for exact source validation.

10.6 Synthesizer contract

Input: verified claims, citations, contradictions, gaps, original question. **Output:** concise answer/report that distinguishes (a) direct document statements, (b) synthesis/inference, (c) conflicts, and (d) unknowns. It must not introduce a new factual assertion that lacks supporting evidence.

10.7 Prompt-injection resistance

- Clearly delimit retrieved document content as untrusted evidence, not instructions.
- System prompts define policy; documents cannot override tools, authorization, output schema, or safety rules.
- Tool calls are allowlisted and validated with Pydantic/JSON schemas.
- No shell, arbitrary Python, unrestricted network, or direct database query tool is exposed to LLM agents.
- Red-team documents that say “ignore previous instructions,” request secrets, or attempt cross-case access.
- Never let the LLM decide whether the caller is authorized; enforce authorization in backend code.

11. Answer format and citation rules

11.1 Answer structure

- Direct answer or short conclusion, qualified to match the evidence.
- Supporting evidence, each with document title and page/section reference.
- Conflicts or alternative interpretations when present.
- Limitations: OCR quality warnings, missing pages, incomplete documents, or insufficient evidence.
- Follow-up question or specific missing document when required.

11.2 Citation integrity

Every material factual claim should link to one or more source spans. Citation IDs must resolve to a real document and page anchor in the authorized case. Do not fabricate page numbers. If a parser cannot reliably map text to a page, disclose that limitation and cite the closest verifiable anchor rather than pretending precision.

11.3 Insufficient evidence behavior

The system should return a useful abstention, for example: “I could not confirm a termination notice period from the documents indexed. The uploaded scan of pages 7–8 has low OCR confidence; please upload a clearer copy or review those pages.” It may show what it searched and what is missing, without claiming that a clause does not exist.

12. API design

Endpoint (example)	Purpose	Key controls
POST /api/v1/cases/{case_id}/documents	Create upload metadata or initiate upload	Auth, case access, type/size checks, idempotency
POST /api/v1/documents/{document_id}/process	Start ingestion/indexing job	Authorized document, job deduplication, rate limits
GET /api/v1/jobs/{job_id}	Poll processing state	User/case ownership, safe error summaries
GET /api/v1/cases/{case_id}/documents	List documents and quality status	Case access, pagination
POST /api/v1/cases/{case_id}/ask	Run Agentic RAG query	Auth, budget, authorized scope, structured response
GET /api/v1/answers/{answer_id}	Retrieve saved answer/evidence references if implemented	Case authorization and retention rules
POST /api/v1/documents/{document_id}/reindex	Reprocess with new pipeline/model version	Explicit versioning, rate limit, audit event
DELETE /api/v1/documents/{document_id}	Delete metadata, objects and vectors per retention policy	Confirm intent, authorization, deletion job, reconciliation

Use versioned APIs, explicit request/response schemas, request IDs, consistent error codes, pagination, timeouts, and safe error messages. Do not return stack traces, access tokens, internal prompts, storage credentials, or provider secrets.

13. Minimal data model

Keep the product single-user and private. Use Supabase Auth for identity and private Storage for original documents. Avoid organizations, teams, memberships, or role tables until product scope genuinely requires them.

Table	Purpose / key fields
cases	id, owner_user_id, title, description, created_at, updated_at, status
documents	id, case_id, owner_user_id, storage_path, original_filename, detected_mime, sha256, size_bytes, page_count, document_type, version_group_id, quality_status, quality_report, processing_status, parser_version, created_at
document_chunks	id, case_id, document_id, document_version_id, chunk_index, page_start, page_end, section_path, text_content or protected reference, content_hash, embedding_model, index_status, quality flags
audit_events	id, case_id, actor_user_id, event_type, target_type, target_id, request_id, safe_metadata, created_at
processing_jobs (recommended)	id, document_id, case_id, job_type, idempotency_key, status, attempts, error_code, started_at, finished_at, pipeline_version
answers (optional)	id, case_id, user_id, question, answer_payload, evidence_refs, graph_version, created_at, retention status

Use Row Level Security on user-owned rows. Storage policies must enforce private access. A service-role key, if required for a backend operation, is server-only and must be narrowly used; never expose it to the browser. The backend must independently verify the caller token and ownership even if Next.js already checked it.

14. Authentication, authorization, privacy, and security

- Verify Supabase access tokens using supported verification methods; reject expired, malformed, or invalid tokens.
- For every request, verify case ownership and document membership on the server. Do not trust case_id, document_id, or owner_user_id supplied by the client without checking.

- Use least-privilege credentials, secret managers in production, HTTPS, rotation, and separate dev/prod environments.
- Keep Supabase service-role key, OpenAI key, Pinecone key, and backend base URL server-only.
- Keep original documents in private storage; use short-lived signed URLs only where necessary.
- Minimize prompt payloads and external LLM data transmission; document the data-processing implications of the selected provider/account configuration.
- Do not log full legal documents, full prompts, tokens, signed URLs, or sensitive extracted facts by default.
- Use request IDs and safe metadata for auditability. Define retention and deletion behavior for originals, chunks, vectors, cached artifacts, answers, and backups.
- Use dependency pinning, vulnerability scanning, static analysis, secret scanning, container scanning, and infrastructure-as-code checks in CI/CD.
- Apply file size/page limits, parser timeouts, memory limits, rate limits, and bounded concurrency to reduce denial-of-service risk.

15. Environment configuration

15.1 Frontend .env.example

```
NEXT_PUBLIC_APP_URL=http://localhost:3000
AI_BACKEND_BASE_URL=http://127.0.0.1:8000
NEXT_PUBLIC_SUPABASE_URL=
NEXT_PUBLIC_SUPABASE_PUBLISHABLE_KEY=
```

15.2 Backend .env.example

```
APP_ENV=local
API_PREFIX=/api/v1
SUPABASE_URL=
SUPABASE_PUBLISHABLE_KEY=
SUPABASE_SERVICE_ROLE_KEY=          # optional; server-only if needed
OPENAI_API_KEY=
PINECONE_API_KEY=
PINECONE_INDEX_NAME=legal-evidence
EMBEDDING_MODEL=sentence-transformers/all-MiniLM-L6-v2
EMBEDDING_DEVICE=cpu
MODEL_CACHE_DIR=.cache/huggingface
TEMP_DIR=.tmp
MAX_UPLOAD_MB=50
MAX_PDF_PAGES=1000
AGENT_MAX_STEPS=12
AGENT_MAX_RETRIEVAL_RETRIES=2
AGENT_MAX_LLM_CALLS=12
```

Limits above are starting defaults to tune using real test files, not universal optimal values. Validate configuration at startup, fail clearly when required values are missing, and avoid logging secret values.

16. Performance and reliability

- Use asynchronous job processing for long document ingestion; upload completion should not require OCR and embedding to finish synchronously.
- Keep query-time graph bounded by a wall-clock deadline, LLM-call count, token budget, retrieval retry limit, and concurrency limit.
- Cache deterministic artifacts using content hashes and pipeline versions; invalidate caches when source, parser, OCR, chunker, or model versions change.
- Batch CPU embeddings; benchmark realistic document sizes on the target host and tune worker count to avoid CPU/RAM exhaustion.
- Use timeouts, retries with jitter, circuit-breaker behavior for external providers, and idempotent job transitions.
- Expose stage-level latency and error metrics without exposing document contents.

- Gracefully degrade: if vector search is unavailable, optionally offer lexical search with a clear limitation; if LLM service is unavailable, report temporary failure rather than fabricate an answer.
- Provide progress states such as uploaded, validating, extracting, OCR, chunking, embedding, indexing, ready, needs_review, and failed.

17. Evaluation and quality gates

Agentic RAG is only useful if it performs better than a simpler baseline. Maintain a test set of representative legal questions and expected evidence spans, including dirty-data examples. Compare each new agent, prompt, model, or retrieval change against the baseline.

17.1 Evaluation dimensions

Dimension	Example measurement / test
Ingestion coverage	Percentage of pages extracted or explicitly marked unavailable
OCR quality	Character/word error rate on manually reviewed sample; exact-match rates for dates, amounts, party names
Source provenance	Percentage of extracted facts/chunks with valid document and page anchors
Retrieval recall	Recall@k / evidence recall against labeled relevant source spans
Answer grounding	Percentage of material claims supported by cited evidence
Citation validity	Citation IDs resolve; pages are correct; cited span supports claim
Abstention	System declines when evidence is missing or extraction quality is too low
Contradiction handling	Known conflicts are surfaced instead of silently resolved
Robustness	Tests for duplicate files, version conflicts, tables, OCR noise, rotated pages, missing pages, prompt injection
Latency/cost	p50/p95 time, calls, tokens, cost per query and per document
Security	Cross-case access denial, token validation, prompt injection, secret leakage, malicious file tests

17.2 Required test corpus

- Native-text contract PDF with clean clause headings.
- Image-only scanned contract; mixed native/scanned pages.
- Rotated, skewed, low-contrast, and noisy scans.
- Table-heavy evidence and multi-column legal documents.
- OCR confusions in dates, currency, section numbers, party names, and negation.
- Duplicate copies with different filenames; revised versions with conflicting terms.
- Missing or out-of-order pages; truncated or corrupt files.
- Password-protected and unsupported formats.
- Documents containing prompt-injection text or requests to expose another case.
- Questions whose answers are absent from the supplied evidence, requiring abstention.

17.3 Acceptance criteria

- Every returned citation resolves to an authorized source and valid page/section anchor, or is explicitly marked as a limitation.
- No test can retrieve another user's case content through direct API calls, vector filters, agent tools, or guessed IDs.

- Known contradictory versions are surfaced and not silently merged.
- Low-quality extraction is visibly flagged and causes abstention or qualified output for material claims.
- The graph terminates within configured step/cost/latency budgets, including retry loops.
- All ingestion jobs are idempotent or safely recoverable after interruption.
- The system preserves original files and transformation lineage.
- Agentic version demonstrates measurable improvement on a defined evaluation set versus a simpler hybrid-RAG baseline; otherwise remove unnecessary agent calls.

18. Observability, audit, and debugging

- Use one request_id and trace_id across frontend, backend, graph nodes, jobs, Postgres, and provider calls.
- Record graph version, node name, duration, retry count, tool type, token/cost metrics, safe reason codes, and outcome.
- Keep content logging disabled by default; enable carefully controlled redacted samples only in a development/test environment.
- Persist a compact evidence manifest for answers so a reviewer can see which documents/pages supported the response.
- Record parser/OCR/chunker/embedding/index versions so results can be reproduced or re-evaluated.
- Separate user-facing errors from internal diagnostics; protect logs and traces as sensitive operational data.

19. Deployment architecture

19.1 Local development

1. Run Next.js frontend on localhost:3000.
2. Run FastAPI backend on 127.0.0.1:8000 using backend/.venv and backend/.env.
3. Use a dedicated local .cache and .tmp under backend; ignore both in Git.
4. Connect to a development Supabase project and a development Pinecone index, or use safe local test doubles for unit tests.
5. Verify auth, private upload, one clean PDF, one scanned PDF, indexing, hybrid retrieval, citations, and abstention before enabling the full graph.

19.2 Production

- Deploy frontend and backend independently, with separate build/release pipelines and health checks.
- Configure AI_BACKEND_BASE_URL to the backend's HTTPS endpoint in the frontend server environment.
- Store secrets in the deployment platform's secret manager; never commit .env files.
- Use separate Supabase/Pinecone environments for development and production.
- Use a persistent job mechanism for ingestion; do not rely on ephemeral local filesystem state for durable uploads or job state.
- If deployed on ephemeral containers/serverless platforms, ensure OCR temp files and model cache are handled appropriately; consider a persistent cache volume or image/model preloading where suitable.
- Pin dependency versions and model/index schema versions; run migration and rollback procedures.
- Apply resource limits, autoscaling policies, alerting, backup and restore tests, and incident procedures.

20. Error model and user experience

Code / state	Meaning	User-facing behavior
UNSUPPORTED_FILE	File type not supported	Explain supported formats and offer conversion guidance
FILE_TOO_LARGE	Configured size/page limit exceeded	Show the applicable limit
PASSWORD_REQUIRED	Encrypted file could not be processed	Request a permitted unlocked copy or secure password workflow
EXTRACTION_LOW_QUALITY	Text may be incomplete or inaccurate	Show warning and affected pages; allow re-upload/review
PROCESSING_FAILED	Pipeline failed after bounded retries	Show safe error ID and retry option
INDEXING_PENDING	Processing is not yet searchable	Show progress; do not claim it is indexed
INSUFFICIENT_EVIDENCE	No reliable support found	Explain what is missing; do not guess
CONFLICTING_EVIDENCE	Sources disagree	Show competing claims and citations
AUTHORIZATION_FAILED	Caller lacks access	Return safe denial without leaking whether unrelated records exist
PROVIDER_UNAVAILABLE	External dependency unavailable	Retry safely or show temporary failure

21. Build roadmap

Phase	Build	Exit gate
0. Foundations	Repo layout, env validation, auth, minimal schema, private storage	Unauthorized users cannot access case data; clean local setup documented
1. Reliable ingestion	File validation, native text parsing, OCR routing, quality report, source anchors	Dirty-data test set has explicit statuses; original files remain intact
2. Search baseline	Structure-aware chunking, CPU embeddings, Pinecone, lexical + dense hybrid search	Labeled retrieval evaluation baseline exists
3. Grounded RAG	Evidence packets, answer schema, citations, insufficient-evidence response	Claim support and citation tests pass
4. LangGraph orchestration	Typed state, router, bounded retries, graph tracing and stop conditions	Graph budgets enforced; loops and provider failures tested
5. Specialist agents	Research, facts analysis, contradiction/gap, verifier, synthesizer	Each agent has typed input/output and evaluation; measurable improvement demonstrated
6. Hardening	Security tests, prompt injection tests, load tests, deletion/reconciliation, CI/CD	Security and reliability acceptance gates pass
7. Production rollout	Separate deploys, secrets, monitoring, backup/restore, staged release	Operational runbook and rollback verified

Recommended sequencing: do not start by building every agent. First prove that extraction and retrieval are trustworthy. Then use LangGraph to coordinate targeted retrieval and evidence validation. Add each agent only when it improves measurable quality.

22. Risks and mitigations

Risk	Mitigation
OCR misreads names, dates, money, or negation	Per-page quality flags, targeted review, original image access, explicit uncertainty
Agents hallucinate or reinforce one another	Require source spans, independent verification, deterministic citation checks, abstention
Agent loop causes cost/latency spike	Hard graph-step, call, token, wall-clock, and cost budgets
Prompt injection inside evidence	Treat document text as untrusted data; tool allowlists; no unrestricted tools
Cross-case leakage	Backend authorization on every operation; case filters; RLS; isolation tests
Index and database drift	Idempotent jobs, state machine, reconciliation, delete/reindex workflows
Poor legal conclusions from incomplete record	Scope caveats, conflicting-evidence display, lawyer review, no unsupported assertions
External provider privacy concerns	Minimize payloads, review provider terms/configuration, avoid sensitive logs, document data flows
Too many agents increase complexity without benefit	Benchmark against hybrid RAG; retain only agents with measurable lift

23. Final product behavior checklist

- A lawyer can upload a document privately and see processing status and quality warnings.
- A document with mixed scanned/native pages is routed page by page.
- The system preserves originals, extracted text lineage, and source/page anchors.
- The lawyer can ask questions across a case and receive grounded answers with citations.
- Complex questions can trigger bounded subquestion planning and targeted retrieval retries.
- Contradictions and document-version differences are surfaced, not silently resolved.
- The system identifies insufficient evidence and low-quality OCR rather than guessing.
- Each agent has a clear role, typed output, bounded tools, and evaluation coverage.
- The answer verifier rejects invalid citations and unsupported material claims.
- All backend Python components can be built and deployed independently from frontend code.
- Production URLs and secrets are environment-configured; no production URL is hard-coded.
- No agent can bypass authorization or perform legal/external actions autonomously.

Appendix A. Practical definition of done for Agentic RAG

The system may be called an implemented multi-agent/Agentic RAG application only after the graph, not just the document, actually routes work among the defined specialist roles. Each agent must have a real contract and observable execution; merely naming functions “agents” is not sufficient. A minimal meaningful version has: (1) router/planner, (2) research/retrieval, (3) facts/analysis, (4) evidence verification, and (5) synthesis, with contradiction/gap analysis added for multi-document work. Some stages may be combined at first.

Appendix B. Key implementation rules

- One Python backend deployment can host all graph nodes and agents; no separate agent microservices are required.
- Use LangGraph for orchestration and LangChain components where useful, but keep business rules independent of framework-specific magic.

- Use CPU embeddings by default and benchmark real workloads; CPU support does not imply unlimited throughput.
- Never claim dirty data is handled perfectly. Measure quality, preserve originals, expose warnings, and abstain where necessary.
- Store source provenance with every chunk and fact; the provenance is part of the data model, not just prompt text.
- Treat all uploaded files, OCR output, retrieved passages, and model output as untrusted until validated.
- Use a simpler baseline as a control group. Agentic complexity is justified only if it improves grounded correctness, recall, robustness, or usability.

Appendix C. Glossary

Term	Plain-language meaning
RAG	Retrieve relevant source material, then generate an answer based on it.
Agentic RAG	RAG inside a controlled workflow that can plan, retry searches, inspect evidence, validate, and stop when evidence is weak.
Agent	A bounded component that performs a defined task, often using an LLM and approved tools.
LangGraph	A framework for defining stateful workflows with nodes, conditional edges, and controlled loops.
Hybrid retrieval	Combines semantic vector search with keyword/lexical search.
Provenance	Information showing exactly which original document, page, section, and processing path produced a fact or chunk.
Abstention	Explicitly declining to make a claim when evidence is insufficient.
Dirty data	Real-world files or extracted text that are incomplete, noisy, malformed, duplicated, inconsistent, or ambiguous.

Appendix D. Reference configuration summary

Frontend: Next.js, React, TypeScript; independent Vercel/equivalent deployment. Backend: Python FastAPI, LangChain, LangGraph, OpenCV, OCR/parser libraries, sentence-transformers. Data: Supabase Auth/Postgres/private Storage; Pinecone vectors. LLM: OpenAI API. Embeddings: sentence-transformers/all-MiniLM-L6-v2 on CPU. API: versioned REST under /api/v1. Authorization: Supabase token verification plus server-side case/document ownership checks. User model: single lawyer/private workspace.

Implementation note: choose exact OCR/parser libraries after testing a representative corpus, licensing, language coverage, deployment constraints, and accuracy. This specification intentionally defines behaviors and acceptance tests rather than claiming one OCR engine will be best for every document.

End of v4.0 specification